

Agentic Engineering with Claude Code

A Practical Field Guide for Software Engineers and Engineering Leaders

Agentic engineering is the discipline of building production software with an AI agent that explores your codebase, plans changes, edits files, runs commands, and verifies its own work — while you stay in control of what ships. It is not autocomplete, and it is not "ask a chatbot for a snippet and paste it in." It is a different operating model in which you describe outcomes, the agent does the mechanical work, and your time shifts from typing code to specifying intent, shaping context, and reviewing results.

This guide is written for people who already know how to ship software and now want to introduce agentic practices into a real product team without lowering the quality bar. It is deliberately practical: concrete prompts, end-to-end workflows, dos and don'ts, the mistakes everyone makes, and a phased adoption plan. It draws on Anthropic's published best practices for Claude Code, on Andrej Karpathy's 2025 talk "*Software Is Changing (Again)*", on the agentic-development themes that dominated Google I/O 2025, and on Dave Rensin's Elephant-Goldfish Model for spec-first development. Sources are listed at the end.

Part 1 — The mental model that makes everything else work

Before any tooling, get the mental model right. Most failures with agentic coding are not tool failures; they are model-of-the-tool failures.

Treat the agent as a fast, capable, forgetful partner — not a magic code generator

The single most useful reframing is to treat the agent like a strong junior engineer who has tools, can read your whole repo, works extremely fast, and has no long-term memory. That framing predicts its behavior well: it will do impressive multi-file work in seconds, and it will also confidently do the wrong thing if you under-specify, lose the thread in a long session, or never give it a way to check itself.

Karpathy made the same point from the opposite direction in his 2025 YC talk. He described today's models as having "jagged intelligence" — capable of solving genuinely hard problems one moment and failing trivial ones the next — and likened them to a brilliant colleague with anterograde amnesia: no durable memory beyond the current context window. The practical implication is that **you** supply the memory, the judgment, and the verification. The agent supplies speed and breadth.

Keep the AI on a leash

Karpathy's most-quoted line from that talk is a warning that the industry was getting "way over-excited" with autonomy, and that the right move is to *keep AI on a leash*. His reasoning is concrete and engineering-grade: a 10,000-line diff that lands instantly is not a gift, because **you are still the bottleneck** — you have to confirm it introduces no bugs, no security holes, and actually does the right thing. A diff you cannot review is a liability no matter how fast it arrived.

This translates into four operating rules that recur throughout this guide:

- **Smaller tasks** beat one giant task.
- **Clearer prompts** beat vague ones.
- **Narrower scope** beats open-ended scope.
- **Incremental changes** beat big-bang changes.

Discipline is not removed by AI; it is *rewarded* by it. The teams that win are the disciplined ones moving fast, not the reckless ones moving faster.

The generator-verifier loop is the unit of work

Think of every task as a loop: the agent **generates** a change, then something **verifies** it. The whole game is making that loop fast and trustworthy. Karpathy's framing — and Anthropic's — converge here: if there is no automatic verifier (a test, a build, a linter, a screenshot diff), then *you* are the verifier, and every mistake waits for a human to notice it. Give the agent a check it can run and the loop closes by itself: the agent does the work, runs the check, reads the result, and iterates until it passes.

The autonomy slider: suits, not robots

Karpathy uses the Iron Man analogy: a *suit* augments a human who stays in the loop; a *robot* operates alone. Most useful agentic work today should look like a suit. Picture autonomy as a slider you set per task based on how reversible and how verifiable the work is:

Autonomy level	When to use it	Example
Tight (watch every step)	Unfamiliar code, risky or irreversible changes, security-sensitive paths	Touching auth, migrations, payment flows
Medium (plan → approve → run)	Clear-ish scope, multi-file, you can review the plan	Adding a feature behind a flag
Loose (run, then review the diff)	Well-scoped, well-tested, easily reversible	Lint fixes, mechanical refactors, test scaffolding
Unattended (with hard gates)	Repetitive batch work with a deterministic pass/fail check	Migrating 2,000 files one at a time

The skill is choosing the right level — and dialing it down the instant you are surprised by the output.

Part 2 — How Claude Code actually works (the parts that change your behavior)

You do not need the internals, but three facts shape every good habit.

It runs an agentic loop with real tools. Claude Code reads files, runs shell commands, edits across many files, and integrates with your existing tools — in your terminal, IDE, desktop app, or browser. It is not generating text for you to apply; it applies changes and observes the results.

The context window is your fundamental constraint. The context window holds the entire conversation: every message, every file the agent reads, every command's output. It fills up fast — a single debugging session can burn tens of thousands of tokens — and model performance degrades as it fills. When it gets full, the agent starts "forgetting" earlier instructions and making more mistakes. *Managing context is the core skill of agentic engineering.* Nearly every best practice in this guide is downstream of this one fact.

It asks permission before doing risky things — and you tune that. By default the agent requests approval for actions that can modify your system (file writes, many shell commands, MCP tools). This is deliberately conservative. You decide how much of that friction to remove, trading clicks for autonomy as your trust in a task grows.

Part 3 — Configure your environment once; benefit on every task

A small amount of setup makes every future session sharper. Do this at the repo and team level, not ad hoc.

CLAUDE.md: the project's persistent memory

`CLAUDE.md` is a file the agent reads at the start of every conversation. It is where you put the context the agent cannot infer from code alone: the commands it can't guess, the conventions that differ from defaults, the gotchas that bite newcomers.

Generate a starter with the `/init` command (it inspects your build system, test framework, and patterns), then refine it over time. Keep it short and human-readable. A useful test for every line: *"Would removing this cause the agent to make a mistake?"* If not, cut it. A bloated `CLAUDE.md` is worse than a short one, because important rules get lost in the noise and the agent starts ignoring half of it.

A good starting skeleton:

```
# Project: Acme Payments API

## Commands
- Build: `make build`
- Test (single file, preferred): `pytest path/to/test_file.py -x`
- Lint + typecheck: `make check` ← run this after any series of edits

## Conventions
- Use dependency injection for all external services (see `src/di/`)
- Money is always `Decimal`, never float. No exceptions.
- API routes: kebab-case URLs, camelCase JSON, version in path (/v1/)

## Gotchas
- `LegacyBillingClient` retries internally; do NOT add an outer retry loop
- Migrations require `ENABLE_DDL=1` in the local env

## Repo etiquette
- Branch names: `feat/...`, `fix/...`, `chore/...`
- Conventional commit messages; one logical change per PR
```

What belongs here vs. what does not:

Include	Exclude
Commands the agent can't guess	Anything it can learn by reading the code
Style rules that differ from language defaults	Standard conventions it already knows
Preferred test runner and how to run a single test	Full API docs (link to them instead)
Branch/PR conventions	Information that changes frequently
Architectural decisions specific to your project	File-by-file descriptions of the codebase
Non-obvious behaviors and gotchas	Platitudes like "write clean code"

Practical tips that compound: press `#` during a session to have the agent append an instruction to the right `CLAUDE.md`; emphasize the rules that matter with "IMPORTANT" or "YOU MUST" to improve adherence; and **check `CLAUDE.md` into git** so the whole team contributes and benefits. Treat it like code — review it when behavior drifts, and prune it regularly. Monorepos can layer files: a root `CLAUDE.md` for global principles and per-package files for local constraints, which mirrors how senior engineers actually think (global rules plus local context). Keep personal, machine-specific notes in `CLAUDE.local.md` and gitignore it.

Skills, hooks, MCP, and subagents — match the tool to the job

`CLAUDE.md` loads every session, so it is the wrong home for things that only matter sometimes. Use the right extension instead:

- **Skills** (`.claude/skills/<name>/SKILL.md`) give the agent domain knowledge or a reusable workflow that it loads *on demand* when the task matches, so they don't bloat every conversation. Use them for "how we do X" knowledge (API conventions, our deployment ritual) and for repeatable procedures you want to invoke by name. Add `disable-model-invocation: true` to a skill with side effects so it only runs when you trigger it explicitly.
- **Hooks** run scripts deterministically at fixed points in the loop (e.g., after every file edit, or before a turn can end). Unlike `CLAUDE.md` instructions, which are advisory, hooks *guarantee* the action happens. Use them for things that must occur with zero exceptions: run the linter after each edit, block writes to the `migrations/` folder, run the test suite before a turn is allowed to finish. You can even ask the agent to write the hook for you.
- **MCP servers** connect the agent to external systems — your issue tracker, your database, your design tool, your observability platform. This is what turns the agent from a code generator into a tool-orchestrating engineer: it can read a ticket's acceptance criteria, query the data model before writing a migration, or check error patterns before debugging. Connect them with `claude mcp add`. (Notably, MCP — introduced by Anthropic — was singled out at Google I/O 2025 as part of the emerging open agent ecosystem, alongside Google's Agent2Agent protocol.)
- **Subagents** (`.claude/agents/`) run in their *own* context window with their *own* allowed tools, then report a summary back. They are your primary defense against context bloat for anything that reads a lot of files, and they enable fresh-eyes review. A canonical example is a dedicated security reviewer that only has read/grep/bash tools and a sharp prompt.
- **CLI tools** are the most context-efficient way to reach external services. Install `gh` and the agent will open PRs, read

issues, and post comments natively; the same goes for `aws`, `gcloud`, and `sentry-cli`. The agent is also good at learning an unfamiliar CLI: tell it to run `some-tool --help` and then use it.

Permissions and sandboxing — remove friction without losing control

After the tenth approval prompt you are not reviewing anymore, you are clicking through. Three levers reduce that friction while keeping you safe: an **allowlist** of specific commands you know are safe (`npm run lint`, `git commit`); **auto mode**, where a separate classifier model reviews each command and blocks only risky ones (scope escalation, unknown infrastructure, hostile-content-driven actions); and **sandboxing**, OS-level isolation that limits filesystem and network access so the agent can move freely inside a fenced yard. Start strict, loosen deliberately as trust grows, and keep irreversible actions gated.

Part 4 — Structuring tasks and breaking down work

How you decompose work matters more than how you prompt any single step. The most reliable pattern, recommended by Anthropic and consistent with Karpathy's "smaller tasks, narrower scope," has four phases.

Explore → Plan → Code → Commit

1. **Explore.** Put the agent in plan mode (it reads and answers, but makes no changes) and have it build a picture of the relevant code. *"Read `/src/auth` and explain how we handle sessions and login. Also look at how we manage secrets in the env."*
2. **Plan.** Ask for a concrete implementation plan: which files change, what the data/control flow is, what the edge cases are. Read the plan. Edit it directly if needed. This is where you catch "solving the wrong problem" before a single line is written.
3. **Code.** Leave plan mode and let the agent implement against the plan — and crucially, against a verifier: *"Implement the plan. Write tests for the callback handler, run the suite, and fix failures."*
4. **Commit.** Have it commit with a descriptive message and open a PR.

Skip the plan when you could describe the diff in one sentence (a typo, a log line, a rename) — planning has overhead. Use it when the approach is uncertain, the change spans multiple files, or you are unfamiliar with the code. The judgment of *when to plan* is itself part of the craft.

Let the agent interview you, then write a spec

For larger features, the highest-leverage move is to have the agent extract the spec from your head before it codes. Start minimal and ask it to interview you:

```
I want to build [one-line description]. Interview me in detail using the
AskUserQuestion tool. Ask about technical implementation, UI/UX, edge cases,
concerns, and tradeoffs. Skip the obvious; dig into the hard parts I might
not have considered. Keep going until we've covered everything, then write a
complete spec to SPEC.md.
```

A good spec is self-contained: it names the files and interfaces involved, states what is explicitly *out of scope*, and ends with an end-to-end verification step that proves the feature works. Then start a **fresh session** to implement it — clean context, focused entirely on the build, with the written spec as the source of truth. Karpathy's observation that "taste, judgment, and spec design remain the human bottleneck" is exactly why this step pays off: time spent making the spec precise beats time spent watching the implementation.

Harden big specs before you build: the Elephant–Goldfish Model

For large or architectural features, upgrade the interview pattern to the **Elephant–Goldfish Model (EGM)**, published in 2026 by Dave Rensin, a Distinguished Engineer at Google, after a year of building an internal tool that grew past 100,000 users on AI-assisted contributions. The metaphor: the **Elephant** is the long-lived, context-rich design session and the design document it produces — it accumulates the institutional knowledge, the micro-decisions, and the arguments behind a feature. The **Goldfish** is a brand-new session with zero memory that knows only what you put in front of it. The discipline in one line: grow the doc with the Elephant, then prove the doc with Goldfish.

The reasoning behind it is an observation worth internalizing: documents are orders of magnitude smaller than code, so a well-written design doc is a cheaper, faster, and more reliable context channel than making an agent interpret thousands of lines of source. And when agents write most of the code, the micro-decisions humans used to leave in the

code itself vanish — unless you force them to shift left into a document a human can actually stand behind. Rensin's phrase for the end state: "design is the new code."

Phase 1 — grow the Elephant (no code yet). Load context (point the session at the relevant docs and code, have it explain the system back, correct it immediately), then hold a genuine design debate. Two of his tactics improve on the basic interview. First, make the *agent* propose the first draft design in prose and block diagrams — if it understands your system, the proposal shows it; if you propose first and ask for a reaction, your blind spots survive. Second, fight sycophancy explicitly: when the session collapses into agreement, tell it that challenging your thinking is the only helpful behavior, and ask "why do you think that?" to snap it out of confident nonsense. In Claude Code, plan mode is the enforced version of his "no code" rule — the agent physically can't start coding while you debate. Don't rush this; on big features the debate is the work.

Phase 2 — write the doc, section by section (never one-shot). The EGM template extends our SPEC.md structure:

1. **Problem** — a plain-English statement a casual reader can follow.
2. **Technical plan** — prose and a block diagram of the major components.
3. **Alternatives considered and rejected** — the underrated section. Documented rejections become guardrails: the implementing agent can't hallucinate its way back into an approach you already ruled out, because the ruling is *in its context*.
4. **Detailed implementation** — every file that will be created or changed, and why. This is what keeps a long implementation on the rails.
5. **Verification** — our addition, to stay consistent with the verifier-first habit: the end-to-end check that proves the feature works.

Phase 3 — the Goldfish gate. Before any code, test the *spec itself* against fresh eyes. A subagent is a goldfish by construction, and headless mode makes the three checks one-liners:

```
# Goldfish 1 — comprehension: is the doc self-sufficient?
claude -p "Read SPEC.md and only the files it references. Explain what it
is trying to accomplish and how the current system works. If anything is
unclear or missing, list it."

# Goldfish 2 — critic: what did we miss?
claude -p "You are a skeptical technical reviewer. Read SPEC.md and its
referenced files. List faulty assumptions, missing edge cases, and
ambiguities. Flag only what affects correctness."

# Goldfish 3 — readiness: could a newcomer build it in one pass?
claude -p "You are an engineer new to this codebase. Read SPEC.md. Could
you implement this in a single pass? List every question you would need
answered first."
```

If the comprehension goldfish can't explain the system from the doc alone, the doc is missing context — fix the doc, not the goldfish. Expect roughly a third of the critic's findings to be genuinely valuable; act on the correctness ones and skip the rest, exactly as with any fresh-eyes reviewer. Loop until the findings degrade into nitpicks, then get a human sign-off. Wrap the three checks as a `/harden-spec` skill and the gate becomes one command for the whole team.

Phase 4 — implement. Hand the hardened doc to your normal implementation loop. A side benefit: recovery from a crashed or polluted session becomes a 30-second operation — open a fresh session, "read SPEC.md and the files it references, tell me where we are, continue."

Three calibrations. First, EGM is the heavyweight end of the rigor dial — multi-day design debates and per-file enumeration pay for themselves on architectural changes and big features, and are pure overhead on a one-sentence diff; keep matching rigor to task size. Second, EGM hardens the *input* while the verification loop in Part 8 hardens the *output* — they compose, they don't compete. Third, commit hardened specs rather than deleting them: an accumulating library of problem statements, rejected alternatives, and acceptance criteria becomes queryable institutional memory ("why did we decide X?") and an onboarding ramp — Rensin's bar for a new contributor is simply a first spec that passes the Goldfish gate.

One more EGM trick maps directly onto the CLAUDE.md hierarchy: to bootstrap an agent into a large legacy codebase, generate README.md files recursively from the leaves of the tree upward — each engineer spends minutes correcting the AI drafts for their directories, then summaries roll up level by level. In about a week even a big repo has a compressed, layered context map that a fresh session can ingest almost instantly, for a fraction of the tokens of reading source. It is the same global-plus-local layering as root and per-package CLAUDE.md files, built bottom-up.

Right-size every task

The failure mode is asking for too much at once. Good task sizing:

- One feature, one PR, one logical change — not "build the whole billing module."
- A diff you can actually read in one sitting. If you can't review it, you can't ship it.
- A clear, stated definition of done that includes how it will be verified.

Part 5 — A library of example prompts

The difference between a vague prompt and a specific one is the difference between many corrections and few. Below, each pair shows the weak version and the strong version.

Scope the task. - ✗ `add tests for foo.py` - ✓ `Write tests for foo.py covering the edge case where the user is logged out. Avoid mocks — use the real in-memory store. Run them after writing and fix failures.`

Give verification criteria up front. - ✗ `implement a function that validates email addresses` - ✓ `Write a validateEmail function. Test cases: "user@example.com" → true, "invalid" → false, "user@.com" → false. Run the tests after implementing and iterate until they pass.`

Point to existing patterns instead of describing from scratch. - ✗ `add a calendar widget` - ✓ `Look at how existing widgets work on the home page; HotDogWidget.php is a good example. Follow that pattern to build a calendar widget that lets the user pick a month and paginate forward/back by year. Don't add new libraries — use what the codebase already uses.`

Debug by symptom, location, and definition of fixed. - ✗ `fix the login bug` - ✓ `Users report login fails after session timeout. Check the auth flow in src/auth/, especially token refresh. First write a failing test that reproduces it, then fix the root cause — don't suppress the error.`

Verify UI changes visually. - ✗ `make the dashboard look better` - ✓ `[paste screenshot] Implement this design. Then take a screenshot of the result, compare it to the original, list the differences, and fix them.`

Ask the codebase questions (onboarding). - ✓ `How does logging work in this service?` - ✓ `What edge cases does CustomerOnboardingFlowImpl handle?` - ✓ `Why does this call foo() instead of bar() on line 333? Check the git history if needed.`

Investigate without polluting your main context. - ✓ `Use subagents to investigate how our auth system handles token refresh and whether we already have OAuth utilities I should reuse. Report back a summary.`

Review in a fresh context. - ✓ `Use a subagent to review the rate-limiter diff against PLAN.md. Check that every requirement is implemented, the listed edge cases have tests, and nothing outside scope changed. Report gaps that affect correctness — not style preferences.`

Use the right autonomy for batch work. - ✓ `Migrate $file from React to Vue. Return only "OK" or "FAIL: <reason>".` (run per-file in a loop with scoped tools)

A note on vague prompts: they are not always wrong. "What would you improve in this file?" is a great *exploratory* prompt precisely because it surfaces things you wouldn't think to ask. Use vagueness on purpose, when you can afford to course-correct — not by accident on work that has to be right.

Part 6 — Example agentic workflows, end to end

Before the individual workflows, here is the whole system on one canvas — the working loop that the workflows below instantiate in different ways:

Agentic Engineering — the working loop

spec → implement → verify → iterate → ship (human on the leash)

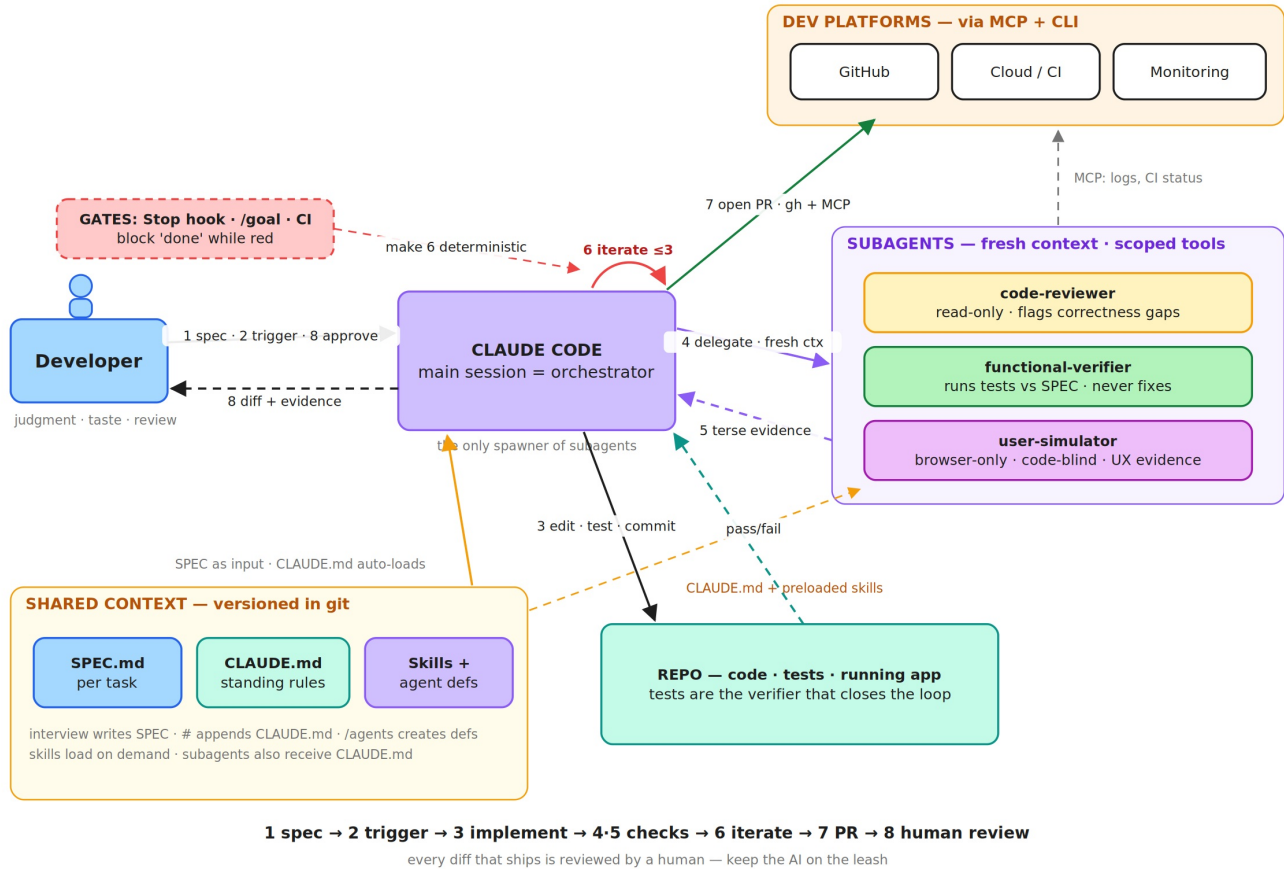


Figure — The working loop. Read it by the numbers: the developer and the main session co-write the spec (1) and the developer triggers the run (2); the main session implements against the repo, whose tests return the pass/fail signal (3); it delegates checks to subagents in fresh contexts (4) which return terse evidence (5); it iterates until all checks pass in the same round (6), opens a PR (7), and hands the diff plus evidence back to the human for review (8). Two structural rules are baked in: the main session is the only thing that can spawn subagents, and gates (Stop hooks, /goal, CI) make "done" deterministic rather than advisory. The shared context layer — SPEC.md, CLAUDE.md, skills, and agent definitions, all versioned in git — feeds both the main session and the subagents.

Workflow A — Ship a feature

1. **Interview → SPEC.md** (Part 4). Lock the scope and the verification step.
2. **Fresh session, plan mode.** "Read SPEC.md and the files it names. Produce an implementation plan." Edit the plan.
3. **Implement against tests.** "Implement the plan behind the `oauth_login` flag. Write tests for the callback handler. Run `make check` and the suite; fix failures. Show me the test output."
4. **Adversarial review** in a subagent against the spec.
5. **Commit + PR**, then a human reviews the diff and the evidence.

Workflow B — Fix a production bug

1. **Reproduce first.** "Write a failing test that reproduces this symptom: [paste error/trace]. Don't fix anything yet."
2. **Root-cause.** "Now find the root cause. Explain it before changing code." (Optionally use MCP to read the Sentry issue or query the DB.)
3. **Fix + verify.** "Fix the root cause and make the failing test pass without weakening other tests. Show the before/after test output."
4. **Guard against regression.** Keep the new test; consider a hook or CI check if the class of bug recurs.

Workflow C — Large refactor or migration (fan-out)

This is where agentic engineering massively outpaces hand work — if there is a deterministic check.

1. **Enumerate.** "List all 2,000 Python files that need migrating; write them to files.txt."

- 2. **Pilot on 2-3 files** interactively. Refine the prompt based on exactly what goes wrong.
- 3. **Run unattended at scale**, one file per invocation, with scoped permissions and a pass/fail signal:

```
for file in $(cat files.txt); do
  claude -p "Migrate $file from React to Vue. Return OK or FAIL: <reason>." \
    --allowedTools "Edit,Bash(git commit *)"
done
```

- 4. **Triage the FAILs** by hand. The agent did the 95%; you do the interesting 5%. This mirrors Karpathy's point that the human's job concentrates in the hard, non-verifiable tail.

Workflow D — Writer / Reviewer (two contexts)

Quality improves when the reviewer is not the author, because a fresh context isn't biased toward code it just wrote.

Session A (Writer)	Session B (Reviewer)
"Implement a rate limiter for our API endpoints."	
	"Review @src/middleware/rateLimiter.ts for edge cases, race conditions, and consistency with our middleware patterns."
"Here's the review feedback: [paste]. Address these issues."	

The test-first variant: one session writes the tests from the spec; a second writes code to pass them. Neither can quietly redefine "correct."

Workflow E — Onboard to an unfamiliar codebase

Point the agent at the repo and ask the questions you'd ask a senior engineer: how requests flow, where to add an endpoint, what a confusing function does, what edge cases a class handles. This is one of the fastest ramp-up tools available and it reduces the interrupt load on your teammates.

Workflow F — Unattended runs with hard gates

For work you want to walk away from, don't rely on "looks done." Set a `/goal` condition that a separate evaluator re-checks after every turn, or wire a **Stop hook** that runs your check as a script and refuses to let the turn end until it passes. Combine with auto mode for background safety. The prompt-level "run the check and iterate" works for attended sessions; gates are what make *unattended* sessions finish correctly.

Workflow G — Orchestrate a multi-agent loop

This is the figure at the top of this Part, written out as a runnable workflow. The main session plays developer and conductor (subagents cannot spawn subagents, so orchestration always lives there), dispatching three checking subagents whose contracts are encoded in their tool grants: a **code-reviewer** with read-only tools, so it physically cannot "helpfully" fix things while judging; a **functional-verifier** that runs tests and exercises acceptance criteria but is instructed never to fix; and a **user-simulator** that gets browser tools via its own scoped MCP server, receives *only* the user story and a URL, and is deliberately code-blind — so it grades the product, not the implementation's intentions.

The loop itself is a prompt:

```
Implement the feature in SPEC.md, then run this loop (max 3 iterations):

1. Use the code-reviewer subagent on the diff vs main. Fix only findings
   that affect correctness or the spec; treat the rest as optional.
2. Use the functional-verifier subagent against every acceptance criterion.
   Fix failures at the root cause.
3. Use the user-simulator subagent, giving it ONLY the user story from
   SPEC.md and http://localhost:3000. Fix anything that blocks a user
   from completing a task.

Done = all three report clean in the SAME iteration. Show me each agent's
evidence. If still failing after 3 iterations, stop and summarize what's
unresolved rather than continuing.
```

Three details in that prompt are load-bearing: the **iteration cap** (a reviewer told to find gaps will always find some — without a cap you loop forever into over-engineering), the **same-iteration rule** (a fix for the verifier can break what the

reviewer approved, so all gates must be green simultaneously), and **evidence, not assertions**. Drop the loop into a `/ship-feature` skill so it is one command for the whole team, and add a Stop hook or `/goal` when you want to run it unattended. One context note: every subagent's report lands in the orchestrator's window, so instruct each agent to return a terse, structured report — protect the conductor's context and the loop stays sharp across all three iterations.

Part 7 — Managing context: the core skill

Everything degrades as context fills, so manage it aggressively and deliberately.

- **`/clear` between unrelated tasks.** The "kitchen sink" session — task A, then an unrelated question, then back to A — pollutes context with irrelevant tokens. Reset it.
 - **After two failed corrections, stop and reset.** If you've corrected the agent twice on the same issue, the context is now full of failed approaches dragging it down. `/clear` and write a better initial prompt that bakes in what you just learned. A clean session with a sharper prompt almost always beats a long session with accumulated corrections.
 - **Push investigation into subagents.** Reading hundreds of files to "investigate X" will eat your main context. Delegate it; get back a summary.
 - **Compact with intent.** When you approach the limit, auto-compaction summarizes what matters. For control, use `/compact Focus on the API changes`, and instruct in `CLAUDE.md` what must survive compaction (e.g., "always preserve the full list of modified files and the test commands").
 - **Use checkpoints to take risks cheaply.** Every prompt creates a checkpoint; `Esc Esc` or `/rewind` restores conversation, code, or both. This lets you tell the agent to try something bold and simply rewind if it fails. (Checkpoints track only the agent's changes — they are not a substitute for git.)
 - **Treat sessions like branches.** Name them (`/rename oauth-migration`), resume with `claude --continue` or `claude --resume`. Each workstream keeps its own clean context across days.
 - **`/btw` for throwaway questions** that you don't want occupying context.
-

Part 8 — Reviewing, testing, and validating AI-generated code

This is where engineering judgment lives, and where leaders should set firm norms. Karpathy's "I'm still the bottleneck" is not a complaint — it is the job description. The goal is to make verification *fast and trustworthy*, not optional.

Make the agent prove it, not assert it

"Looks done" is the weakest possible signal. Require **evidence**: the test output, the exact command run and what it returned, the screenshot compared against the design. Reviewing evidence is faster than re-running everything yourself, and it works even for sessions you didn't watch. A flat rule worth adopting: **if you can't verify it, don't ship it**.

Layer your verification

1. **Automated checks the agent runs in-loop** — tests, build exit code, linter, type checker, a script that diffs output against a fixture, a browser screenshot vs. the design. These close the generator-verifier loop without you.
2. **Deterministic gates** — a Stop hook that blocks the turn until the suite passes; a `/goal` condition an evaluator re-checks each turn. Use these for unattended work.
3. **Adversarial review in a fresh context** — a subagent that sees only the diff and the criteria, not the reasoning that produced it, so it grades on the result's own merits. Run the bundled `/code-review` skill for a correctness pass, or write a custom review prompt to check the diff against the plan.
4. **Human review of the diff and the evidence** — always, for anything that ships. You are accountable for the code regardless of who typed it.

Watch for over-engineering from your reviewers

A reviewer asked to find gaps will almost always find some, even when the work is sound — that's what it was told to do. Chasing every finding leads to needless abstraction, defensive code, and tests for impossible cases. Tell reviewers to flag only what affects correctness or stated requirements and to treat the rest as optional. Apply human judgment to the findings list, not just to the code.

Security gets its own pass

Use a dedicated security-reviewer subagent (read/grep/bash only) prompted to hunt injection, authn/authz flaws, secrets in code, and unsafe data handling, with line references and suggested fixes. Never relax permissions on security-sensitive paths, and keep secrets out of prompts and out of the repo.

Tests are the currency of trust

The stronger your tests, the higher you can safely turn the autonomy slider. Insist that AI-written features come with AI-written tests *that you have read* — and be alert to the classic move of "fixing" a failure by weakening an assertion. The test-first and writer/reviewer workflows exist precisely to make tests harder to game.

Part 9 — Using repositories effectively

- **Git worktrees for parallelism.** Run multiple sessions in isolated checkouts so their edits never collide. This is what makes "three agents working at once" practical rather than chaotic. The desktop app manages these visually; Claude Code on the web runs sessions in isolated cloud VMs; agent teams coordinate multiple sessions with shared tasks.
- **Headless mode in CI and hooks.** `claude -p "..."` with `--output-format json` (or `stream-json --verbose`) integrates the agent into pipelines, pre-commit hooks, and scripts. Pipe results into other commands: `claude -p "<prompt>" --output-format json | your_command`.
- **Branch and PR hygiene stays human-defined.** Put your conventions in `CLAUDE.md`; require conventional commits and one logical change per PR; have the agent open PRs via `gh` but keep merge authority with people.
- **Commit `CLAUDE.md`, skills, agents, and hooks.** Your `.claude/` directory is shared infrastructure. When one engineer teaches the agent something, the whole team should inherit it.

Part 10 — Dos and don'ts

Do - Give the agent a way to verify its work on every task. - Explore and plan before coding anything non-trivial. - Be specific: name files, reference patterns, state constraints and the definition of done. - Keep `CLAUDE.md` short, true, and version-controlled. - `/clear` between unrelated tasks; reset after two failed corrections. - Use subagents for investigation and for fresh-eyes review. - Right-size tasks to diffs you can actually read. - Match autonomy to reversibility and verifiability. - Read every diff that ships and require evidence, not assertions.

Don't - Don't accept a diff you can't review just because it arrived fast. - Don't let one session sprawl across unrelated tasks. - Don't pile correction on correction in a polluted context. - Don't stuff `CLAUDE.md` with things the agent can already infer. - Don't skip the plan on multi-file or unfamiliar changes. - Don't ship without a verifier; "looks done" is not done. - Don't relax permissions on auth, migrations, payments, or anything irreversible. - Don't put secrets in prompts or in the repo. - Don't outsource your judgment — taste, architecture, and security stay yours.

Part 11 — Common mistakes and how to avoid them

- **The kitchen-sink session.** Context fills with unrelated work and quality drops. → `/clear` between tasks.
- **Correcting over and over.** The context is now polluted with failed attempts. → After two failed corrections, `/clear` and write a sharper initial prompt incorporating what you learned.
- **The over-specified CLAUDE.md.** It's so long the agent ignores half of it. → Prune ruthlessly; if the agent already does something right, delete the rule or convert it to a hook.
- **The trust-then-verify gap.** A plausible implementation that quietly mishandles edge cases. → Always provide verification; if you can't verify it, don't ship it.
- **The infinite exploration.** "Investigate this" with no scope reads hundreds of files and floods context. → Scope investigations narrowly, or push them into a subagent.
- **Big-bang tasking.** Asking for an entire subsystem in one prompt yields an unreviewable diff. → Decompose into one-PR-sized chunks.
- **Skipping the spec on hard features.** The agent solves the wrong problem fast. → Interview → `SPEC.md` → fresh session.

- **Over-automating too early.** Turning every gate off before you trust a task. → Loosen autonomy deliberately, one notch at a time, never on irreversible paths.
- **Reviewer-driven over-engineering.** Chasing every gap a fresh-eyes reviewer reports. → Only act on findings that affect correctness or requirements.

Part 12 — Recommended team practices

Agentic engineering is a team sport once more than one person is doing it. Standardize the things that benefit from being shared and keep the rest flexible.

- **Shared `.claude/` infrastructure.** Maintain a reviewed, version-controlled `CLAUDE.md`, a library of team skills (your API conventions, deploy ritual, common workflows), reusable subagents (security reviewer, test writer), and hooks for non-negotiables. Treat changes to these like code changes, with review.
- **A "definition of done" that includes verification.** A task isn't done because the agent says so; it's done when the stated checks pass and a human has reviewed the diff and the evidence.
- **Code-review norms for AI-generated code.** Review the *diff*, not the chat transcript. Require evidence in the PR. Optionally label AI-generated PRs so reviewers calibrate attention. The author of record — and the accountability — is the human who opened the PR.
- **A permissions and sandboxing policy.** Decide as a team which commands are allowlisted, when auto mode is acceptable, when sandboxing is required, and which paths are always gated (auth, migrations, payments, infra).
- **A security posture.** No secrets in prompts or repos; a standard security-review step for sensitive changes; clarity on which MCP connections are approved and what data they can touch.
- **Knowledge sharing.** When someone discovers a prompt pattern, a skill, or a gotcha worth encoding, it goes into the shared `.claude/` directory or a team channel — not into one person's head. Karpathy's "amnesia" point applies to the agent; don't let it apply to your team.
- **Metrics that measure outcomes, not vanity.** Track cycle time, review time, escaped defects, and rework — not "lines of AI code." More AI lines is not a goal; shipping good software faster is.

Part 13 — Adopting agentic engineering gradually

Don't flip a switch across the org. Roll it out the way you'd roll out any practice that touches quality and security: in stages, with guardrails, measuring as you go.

Phase 0 — Foundations (week 1). Pick one motivated team and a low-risk, well-tested service. Install Claude Code, write a first `CLAUDE.md` with `/init`, install `gh`, and agree on a starting permissions policy. Goal: everyone can run the agent and verify its output.

Phase 1 — Augment, don't automate (weeks 2-4). Use the agent as a suit, not a robot. Encourage codebase Q&A for onboarding, explore→plan→code on small features, and bug-fix-with-a-failing-test. Keep autonomy tight and review every diff. Goal: build trust and intuition; notice what makes the agent succeed or struggle.

Phase 2 — Workflows and shared infrastructure (months 2-3). Standardize the writer/reviewer and adversarial-review patterns. Build the first team skills, subagents, and hooks. Introduce verification gates in CI via headless mode. Start a lightweight pattern-sharing habit. Goal: repeatable, reviewable workflows that don't depend on one expert.

Phase 3 — Scale and parallelize (months 3+). Introduce worktrees and parallel sessions for genuinely independent work, and fan-out for large migrations *with deterministic checks*. Pilot longer unattended runs gated by Stop hooks or `/goal`, on reversible work first. Consider async/cloud agents for background tasks. Goal: multiply throughput on the work that is safe to scale, while the hard tail stays with senior engineers.

Throughout: keep irreversible and security-sensitive paths under tight human control, measure outcomes rather than activity, and let teams keep what works and drop what doesn't. The patterns here are starting points, not laws.

Part 14 — Where this is going (and what stays the same)

The direction of travel was unmistakable across Google I/O 2025 and Karpathy's talk, and it informs how you should

invest.

Agents are moving from prototype to product. Google framed agents as systems that combine model intelligence with tool access to take actions *on your behalf and under your control* — the same human-in-the-loop posture Anthropic builds into Claude Code. Google's coding tools made the pattern explicit: **Gemini Code Assist agent mode** acts as an in-IDE pair programmer that proposes a plan, waits for approval, then executes multi-file changes with inline diffs; **Jules** is an asynchronous cloud agent that clones your repo into an isolated VM, works while you do something else, and returns a reviewable PR with its plan, reasoning, and a diff. Different surfaces, identical core loop: *plan* → *approve* → *execute* → *review*.

Synchronous and asynchronous agents are complementary. In-IDE agents (and Claude Code in your terminal) are for work you're actively steering; async/cloud agents are for parallel, walk-away tasks. Expect to use both, and to think about *which mode fits which task* — exactly the autonomy-slider judgment from Part 1.

Context-aware and multimodal is the new normal. I/O's universal-assistant work (Project Astra, Project Mariner) pointed at agents that see screens, read designs, and act across tools. In day-to-day engineering this shows up mundanely and usefully: paste a screenshot and ask the agent to match a design, then have it screenshot its own result and diff it. Multimodal context is just more context — and context is the constraint.

An open agent ecosystem is forming. MCP (from Anthropic) and Agent2Agent (from Google) were both highlighted as connective tissue so agents can reach services and each other. The practical takeaway is to invest in MCP connections to your real systems — issue trackers, databases, observability — because an agent grounded in your actual context outperforms one guessing from code alone.

The reliability gap is real and so is your job. A 95% success rate sounds great until you remember the 5% can be catastrophic. This is the technical reason the human stays in the loop, and the reason Karpathy insists the model is still the *bottleneck* you must guard. The enduring human contributions — taste, architectural judgment, precise specs, and rigorous verification — are not going away. Karpathy's framing is the right note to end on: **vibe coding raised the floor so anyone can build; agentic engineering raises the ceiling for professionals who bring discipline.** The teams that thrive will be the ones that move fast *and* keep the AI on a leash.

Quick reference

Session and context

- `/init` — generate a starter `CLAUDE.md`
- `#` — append an instruction to `CLAUDE.md` mid-session
- `/clear` — reset context between unrelated tasks
- `/compact <focus>` — summarize, preserving what matters
- `Esc` — stop mid-action (context preserved); `Esc Esc` / `/rewind` — restore a checkpoint
- `/btw` — ask a throwaway question without growing context
- `claude --continue` / `claude --resume` / `/rename` — manage sessions like branches

Autonomy and safety

- `/permissions` — allowlist safe commands
- `--permission-mode auto` — classifier-gated autonomy
- `/sandbox` — OS-level isolation
- Stop hook / `/goal` — hard gates for unattended runs

Scale

- `claude -p "..."` `--output-format json` — headless mode for CI/scripts/hooks
- Worktrees / desktop sessions / web / agent teams — parallelism
- `--allowedTools "Edit,Bash(git commit *)"` — scope permissions for batch runs

Spec hardening (big features)

- Elephant: plan-mode design debate → SPEC.md with Problem / Plan / Alternatives rejected / Files / Verification
- Goldfish gate: comprehension → critic → readiness checks in fresh contexts (`claude -p` or subagents), loop until nitpicks
- `/harden-spec`, `/ship-feature` — wrap the gate and the multi-agent loop as team skills

The five habits that matter most

1. Give every task a verifier.
2. Explore and plan before coding anything non-trivial.
3. Be specific; point to files and patterns.
4. Manage context aggressively (`/clear` , subagents, checkpoints).
5. Read every diff that ships — you are accountable for it.

Sources and further reading

- Anthropic, *Best practices for Claude Code* — <https://code.claude.com/docs/en/best-practices>
- Anthropic, *Claude Code overview* and product page — <https://code.claude.com/docs/en/overview> · <https://www.anthropic.com/product/claude-code>
- Andrej Karpathy, "*Software Is Changing (Again)*", Y Combinator AI Startup School, 2025 — <https://www.ycombinator.com/library/MW-andrej-karpathy-software-is-changing-again>
- Dave Rensin, "*Elephants, Goldfish and the New Golden Age of Software Engineering*", 2026 — <https://drensin.medium.com/elephants-goldfish-and-the-new-golden-age-of-software-engineering-c33641a48874>
- Google, *Project Mariner / agents at I/O 2025 keynote* — <https://blog.google/innovation-and-ai/technology/ai/io-2025-keynote/>
- Google, *Gemini Code Assist agent mode* — <https://developers.google.com/gemini-code-assist/docs/agent-mode>
- Google, *Jules: asynchronous coding agent* — <https://blog.google/innovation-and-ai/models-and-research/google-labs/jules/>
- Google, *Gemini as a universal AI assistant (Project Astra/Mariner)* — <https://blog.google/innovation-and-ai/models-and-research/google-deepmind/gemini-universal-ai-assistant/>

Tooling details (commands, flags, file locations) reflect Claude Code as documented in mid-2026 and will continue to evolve; verify specifics against the current official documentation.